

Friction-Guided Inference: Calibrating Correction Strategies and Abstention from Logprob Signals

Tomas Pødenphant Lund

2026-05-23

Independent Research, Aarhus, Denmark.

ORCID: <https://orcid.org/0009-0000-4724-2427>

Correspondence: tomas.lund@frictiontheory.org

Web: <https://frictiontheory.org>

Abstract

Large language models frequently possess the knowledge needed to answer a question correctly yet commit to the wrong response. This paper presents friction-guided inference, a method that uses the model’s own logprob distribution — available at zero cost from any OpenAI-compatible API — to both **select calibrated correction strategies** and **identify questions where the model should abstain rather than commit**.

The key signal is competing routes (CR): the count of high-probability alternative tokens per position. CR detects *when* a model is uncertain (population-level AUC 0.53–0.68) and, through calibration, identifies *which* correction strategies help. An ablation reveals a sharp limit: CR does not reliably determine *which answer is correct* at the individual-question level. The lift comes from the strategy itself — asking the model to reconsider under different conditions — combined with CR-guided uncertainty thresholds that decide when to answer at all.

Headline result: combined strategy + calibrated abstention yields +12 to +21 percentage-point improvements across four of five evaluated cells spanning four architectures (dense transformer, mixture-of-experts, Liquid Neural Networks) and four benchmarks (MATH-500, SimpleQA, MMLU-Pro, GPQA Diamond). Component-wise, the calibrated strategy pipeline alone produces +7.7 to +20.8 pp on held-out data (mean +11.8 pp across four statistically significant cells); entropy-guided abstention alone produces +9.0 to +15.7 pp success-rate improvement at 20% abstention across the four large cells, at zero additional inference cost. A signal ablation (§5.2) shows that the entropy of the logprob distribution is a slightly stronger abstention signal than the discretised competing-routes count, and that no logprob-derived signal detects errors on GPQA Diamond — an honest scope limit. The two mechanisms are complementary —

strategy recovers commitment gaps, abstention prevents confident-wrong commits — and combine super-additively. On SimpleQA, the combined pipeline lifts Qwen3-235B to 57.2% success, surpassing GPT-4o and GPT-4.1 while also expressing calibrated uncertainty where the model should not commit.

The method requires per-model, per-benchmark calibration; the pipeline code and signal-extraction transfer without modification across the four architectures we tested, but we do not claim architectural universality beyond that empirical scope. **Calibration is an online procedure that runs in approximately two hours of API calls at a cost of roughly \$1.50 per cell** — not a hyperparameter search over model weights — but it does require a labelled calibration set of 50–200 questions per cell. It is closer in cost and character to a deployment health check than to model training. All code, data, calibration protocols, and the quality-assurance harness used to produce these results are released, including documentation of bugs discovered and fixed during development.

Table of Contents

Abstract	1
Table of Contents	2
1. Introduction	3
2. Related Work	6
3. Method	8
3.1 The Competing Routes Signal.....	8
3.2 Pipeline Architecture	8
3.3 Per-Model Calibration.....	8
3.4 Commit Mechanisms.....	9
3.5 Quality Assurance	9
4. Results	10
4.1 Summary.....	10
4.2 MATH-500 Level 4–5: Qwen2.5-7B (+20.8 pp)	11
4.3 SimpleQA: Qwen3-235B (+10.6 pp).....	13
4.4 MMLU-Pro STEM: Qwen3-235B (+7.7 pp) and LiquidAI (+8.1 pp).....	14
4.5 GPQA Diamond: GPT-oss-20B (+5.6 pp).....	14
4.6 Cross-Cell Analysis.....	15
5. Discussion.....	16
5.1 Ablation: What Does CR Actually Contribute?	16
5.2 Abstention from logprob uncertainty	18

5.3 The Friction Ceiling	21
5.4 Decomposing strategies: from names to components	22
5.5 Calibration vs. Generalisation	30
5.6 Transfer Across Architectures	31
5.7 Limitations and Methodological Notes	31
5.8 Optional: iterative adaptation as reflexive matching.....	32
6. Conclusion	33
Acknowledgments	33
References	34

1. Introduction

When a large language model answers a factual question incorrectly, it is tempting to conclude that the model lacks the necessary knowledge. Often this conclusion is wrong. In many cases the correct answer appears among the model’s top-ranked token candidates — it was assigned high probability but was narrowly outranked by an incorrect alternative. The paper calls this the *commitment gap*: the correct answer was statistically accessible but the model committed to a marginally more probable wrong one.

We use “know” descriptively: the correct answer was structurally available in the model’s top-k logprob distribution but not committed to. Empirically, this commitment gap is a stable, measurable pattern — retrieval succeeds (the correct token carries substantial probability mass) but commit fails (a competing token wins). It is statistical structure, not arbitrary noise. The practical consequence — which this paper establishes empirically and which does not depend on any particular theory of why the gap exists — is that a commitment gap can be closed by re-prompting the model under different conditions, causing resampling that sometimes lands on the correct alternative. A genuine knowledge gap, where the correct answer has negligible probability, cannot. Under the companion framework (Lund 2026b §9.1b) this pattern is interpreted as a “friction ceiling” arising from race-resolution under bounded resources; that interpretation is offered as context, not as a premise the empirical results below rely on.

This paper presents a calibrated inference pipeline for detecting and closing commitment gaps at inference time using logprob-derived signals. The approach rests on a simple observation: when a model is uncertain, its logprob distribution contains multiple high-probability alternatives — competing routes through the token space. The paper quantifies this as the *competing routes* (CR) signal — a discretised feature derived from the logprob distribution, in the class of logprob-based confidence signals well-studied since Kadavath et al. (2022). For each generated token, CR counts the number of top-k logprob entries with substantial probability mass. High CR indicates that the model was choosing between alternatives. Low CR indicates confident commitment.

Extracting the CR signal is free: it requires only `logprobs=True` in the API call — no additional model calls and no external verifier. This should not be confused with the pipeline being cost-free. The *signal* is a by-product of generation; the *pipeline built on it* still requires per-cell calibration on labelled examples (§3.3), and that labelled calibration set — not the signal — is the real cost of deployment. What the signal does buy is breadth: because CR measures a structural property of probabilistic token generation, the same extraction code works across the architectures, model families, and benchmarks we tested, without per-architecture signal engineering.

Given the CR signal, we build a *calibrated inference pipeline* that:

1. **Calibrates** offline: runs multiple strategies on a small set (50–200 questions), uses CR profiles to identify which strategies are constructive vs. destructive for this model.
2. **Generates** a vanilla response with logprobs.
3. **Applies** the calibrated correction strategy (e.g., step-by-step reasoning, answer verification).
4. **Commits** to the strategy response.

A critical finding from our ablation (§5.1): the lift comes from running the *right strategy*, not from CR-weighted answer selection. Simply trusting the strategy response matches or exceeds CR-guided commit on all cells. CR’s value is in calibration (finding which strategy works) rather than in per-question answer selection.

We evaluate this pipeline on five model–benchmark cells:

Model	Architecture	Benchmark	n (held-out)	Vanilla	Adaptive	Lift	95% CI
Qwen2.5-7B	Dense transformer	MATH-500 L4–5	212	45.8%	66.5%	+20.8 pp	[+15.1, +26.4]
Qwen3-235B	Mixture-of-experts	SimpleQA	3 810	41.1%	51.8%	+10.6 pp	[+9.1, +12.1]
Qwen3-235B	Mixture-of-experts	MMLU-Pro STEM	4 498	55.2%	62.9%	+7.7 pp	[+6.4, +9.0]
LiquidAI LFM2	Liquid Neural Network	MMLU-Pro STEM	4 701	33.8%	41.9%	+8.1 pp	[+7.1, +9.2]
GPT-oss-20B	Dense transformer	GPQA Diamond	148	27.0%	30.4%	+3.4 pp	[+0.0, +6.8]

All results are reported on held-out questions (calibration questions excluded). Four of five cells are statistically significant (bootstrap 95% CIs exclude zero). GPT-oss-20B shows a positive

trend but the CI includes zero ($n=148$). The mean improvement across the four significant cells is +11.8 pp. On SimpleQA, our pipeline pushes Qwen3-235B past both GPT-4o (38.0%) and GPT-4.1 (40.0%).

Four findings deserve emphasis:

The signal transfers across the architectures tested. CR works not only on standard dense transformers (Qwen2.5, GPT-oss) but also on a mixture-of-experts model (Qwen3-235B) and on LiquidAI’s Liquid Neural Network — a non-attention architecture based on continuous-time dynamical systems. The same extraction code, with no per-architecture signal engineering, produces significant lifts on all four models. We frame this as evidence that the signal generalises across the autoregressive LM architectures in our panel, not as a claim of full architectural universality: the MoE model is still transformer-based, and the single Liquid Neural Network result is one model on one benchmark with a low vanilla baseline (\$4.4). Establishing universality would require a wider architecture panel, including stronger non-attention models; we treat it as an open empirical question.

The optimal strategy is model-specific. Step-by-step reasoning is the best correction strategy for Qwen and GPT-oss, but *verification* is best for LiquidAI, and *narrowing* is best for Qwen3 on MMLU-Pro. Challenge-based prompting is destructive on every model except Cogito. This means the pipeline must be calibrated per model — but the calibration protocol is systematic and inexpensive (50–200 questions).

Friction detects difficulty, not correctness. CR reliably indicates that a model is uncertain (wrong answers have higher CR than correct ones at the population level), and this signal drives effective calibration. However, at the per-question level, CR cannot determine *which* of two candidate answers is correct. Ablation shows that simply trusting the strategy response performs as well as or better than CR-weighted answer selection. This is theoretically predicted: friction measures the *cost* of choosing between routes, not which route leads to the right destination (Lund 2026b).

Depth helps but plateaus. Repeating the best strategy up to 20 times reveals a clear oracle ceiling: most retrievable headroom is captured in 2–4 rounds, with diminishing returns thereafter. This ceiling separates *commitment gaps* (recoverable) from *knowledge gaps* (not recoverable by any friction-based method).

The name “friction-guided” reflects the origin of this work in a companion theoretical framework (Lund 2026b, **Friction Theory**), which motivated the search for a logprob-derived signal in the first place. That framework supplies vocabulary and interpretation, and a reader interested in why the signal behaves as it does will find the account there. The present paper, however, is written to stand on its own: every empirical claim below is stated in standard terms — logprob distributions, calibration sets, accuracy lifts, abstention thresholds — and can be evaluated without accepting any claim from the companion paper. Where we use framework vocabulary

(“friction”, “commitment gap”, “race-resolution”), it is signposted as interpretation and is not load-bearing for the results.

All code, data, calibration protocols, audit rules, and pre-flight checks are publicly available at <https://github.com/tplund/friction-theory-p3-friction-guided-inference>.

2. Related Work

Self-consistency (Wang et al., 2023) samples multiple chain-of-thought responses and takes the majority vote. This is conceptually related to our approach but differs in three ways: (1) self-consistency applies the same prompting strategy uniformly, while we select strategies per model via calibration; (2) self-consistency requires 10–40 samples, while we use 1–4; (3) self-consistency has no signal for when to stop sampling or when to abstain.

Chain-of-thought prompting (Wei et al., 2022; Kojima et al., 2022) improves reasoning by eliciting intermediate steps. Our finding refines this: CoT (step-by-step) is constructive on some models and benchmarks but *destructive* on others. On LiquidAI, verification outperforms CoT by a wide margin. Blanket application of CoT is suboptimal — strategy selection must be calibrated per model.

Self-correction (Huang et al., 2024; Madaan et al., 2023) asks the model to revise its own output. A critical survey (Kamoi et al., 2024) shows that LLMs cannot reliably self-correct without external feedback, and exhibit systematic self-bias that amplifies over multiple refinement steps. Our pipeline addresses this by providing an *external* signal (the CR profile) that is independent of the model’s self-assessment. However, our ablation (§5.1) confirms a related finding: CR cannot select the *correct* answer at the per-question level — the external signal detects uncertainty but not truth.

Verifier and reward models (Cobbe et al., 2021; Lightman et al., 2023) train a separate model to score candidate solutions. This requires labelled training data and additional inference cost. CR is free and requires no training.

Logprob-based confidence and entropy. Kadavath et al. (2022) establish that token probabilities correlate with correctness (“language models mostly know what they know”). A comprehensive survey of confidence estimation methods (Geng et al., 2024) categorises approaches into prompting-based, sampling-based, and aggregation-based strategies. Recent work on entropy-guided reasoning (Li et al., 2025) uses Shannon entropy per token as a confidence signal for adaptive depth control — conceptually close to our CR signal. We differ in three ways: (1) CR is a discretised count of competing alternatives rather than continuous entropy, making it more robust to sub-threshold noise; (2) we use CR for strategy *selection* and *abstention*, not for controlling reasoning depth; (3) we demonstrate CR works across architectures including non-transformers.

LLM abstention and selective prediction. A recent survey (Feng et al., 2025, “Know Your Limits”) taxonomises abstention methods into confidence-based, calibration-based, and

training-based approaches. I-CALM (Huang et al., 2026) proposes incentivising confidence-aware abstention to mitigate hallucinations. Token-Entropy Conformal Prediction (TECP; Straitouri et al., 2025) uses entropy-based nonconformity scores with conformal calibration for provable error control. Our approach is simpler: a single CR threshold calibrated on held-out questions, with no conformal machinery. We show that this simple approach yields +6.5 to +14.1 pp success improvement at 20% abstention across all five cells.

Confidence-based routing. Lyu et al. (2024) train confidence tokens to route queries between LLMs of different sizes. We route between *strategies* for the same model — a complementary but distinct application. Our routing is based on offline calibration rather than learned tokens, making it applicable without additional training.

Comprehensive evaluation frameworks (HELM). Liang et al. (2022, 2023) introduced HELM (Holistic Evaluation of Language Models) as a standardised framework for comparing language models against diverse benchmarks under uniform conditions. HELM answers *how do models compare on this benchmark?* by holding evaluation methodology constant across models. The present work complements this at a different level of analysis: rather than comparing models, we calibrate per-model strategies on top of any fixed model, asking *can we improve a given model on a given benchmark by selecting the right inference-time strategy?* The two approaches are not in tension. Our lifts of +5 to +20 pp on MATH-500, SimpleQA, MMLU-Pro, and GPQA Diamond are measured on the same benchmarks HELM uses, but the contribution is methodological rather than comparative. Friction-guided inference can be applied on top of HELM-evaluated models to extract additional performance from any of them at fixed inference cost.

NLP uncertainty and abstention beyond logprob calibration. A broader literature in NLP (Liu et al., 2023; and related work from Yejin Choi’s group) documents that language models systematically under-represent the uncertainty they actually face on commonsense and reasoning tasks. Our *confident-wrong* failure mode — model has low CR but the wrong answer — is the operational signature of this pattern on logprob-based signals. The abstention survey (Feng et al., 2025) catalogues the family of mitigation approaches; what the present framework adds is a structural account of why confident-wrong is structurally invisible to friction-based methods: friction signals computed from logprobs measure the cost of choosing between routes, not which route is correct, and confident-wrong is the regime where the model is not uncertain — it is wrong. This bounds the reachable ceiling for any friction-based selector and points toward orthogonal mechanisms (external verifiers, retrieval-augmented prompting, ambiguity-aware decoding) for the remaining headroom.

Calibration and interpretability of logprob signals. The CR signal is computed directly from the top-k logprobs returned by every OpenAI-compatible API; no calibration training or model surgery is required. This places it in dialogue with the broader calibration and mechanistic-interpretability literature (e.g., Grosse et al., 2023, on influence functions for studying language-model behaviour at Anthropic, and related Anthropic work on how language models represent

uncertainty internally). Our finding that per-token CR correlates with error rate at $\rho \approx -0.42$ across architectures *without any calibration training* is an operational rather than mechanistic result: the underlying logprob distribution carries calibration information the model was not explicitly trained to expose. Whether this generalises to architectures whose logprob interface is more opaque (or unavailable through some commercial endpoints) is a question for future work; the present paper is restricted to models that expose top-k logprobs through their API.

3. Method

3.1 The Competing Routes Signal

For each token in a model’s response, we examine the top-k logprob entries returned by the API. The *competing routes* (CR) count for token t is the number of alternatives with log-probability within a fixed threshold of the top candidate:

$$\text{CR}(t) = |\{i: \log p_i(t) > \log p_1(t) - \tau\}|$$

where τ is a threshold (we use $\tau = \log 5$ throughout, yielding CR values of 1–5 for top-5 logprobs). CR = 1 means the model was confident; CR = 5 means five candidates had comparable probability.

CR is a discretised, thresholded entropy. On paired measurements, CR correlates with Shannon entropy at $r = 0.903$, but CR is more robust: it ignores sub-threshold noise and produces integer values that are directly interpretable as “how many alternatives was the model considering.”

The CR signal is computed by the function `compute_cr_from_logprobs()`, which handles format differences between API providers (OpenAI content arrays, legacy token lists, Ollama top-alternatives). A single implementation works across all providers tested.

3.2 Pipeline Architecture

```
Question → Generate vanilla response + logprobs
           → Compute CR profile
           → Classify friction state
           → Select next strategy from calibrated queue
           → Generate strategy response + logprobs
           → Check stop criterion
           → If not stopped: re-classify, select next
           → Commit via best mechanism (MCW, agreement, mode-of-unique)
```

The pipeline is iterative: after each strategy round, it re-evaluates the CR profile and decides whether to continue or commit. This distinguishes it from fixed-depth approaches like self-consistency.

3.3 Per-Model Calibration

Each model requires calibration before deployment. Calibration consists of:

1. **Breadth calibration:** Run all available strategies (vanilla, step-by-step, challenge, narrow, verify, pre-mortem, reexamine) on 50–200 questions. Classify each strategy as constructive (net positive rescues), destructive (net negative), or neutral.
2. **Depth calibration:** Repeat the best constructive strategy up to 20 times on 50 questions. Find the oracle plateau — the point where additional rounds yield no new correct answers for 3+ consecutive rounds.
3. **Pipeline audit:** An automated analysis that produces a JSON report with recommended strategies, stop criteria, and quality warnings. The audit checks for data quality, judge bias, grading consistency, and oracle plateau. If any check fails, the audit returns status=INCOMPLETE and blocks the adaptive pipeline.

We emphasise that calibration is per model AND per benchmark. The optimal strategy for Qwen3-235B on SimpleQA (step-by-step) differs from its optimal strategy on MMLU-Pro (narrow + pre-mortem + step-by-step). Transferring calibration between cells is explicitly prohibited.

3.4 Commit Mechanisms

After executing 2–4 strategy rounds, the pipeline produces a final committed answer via one of six commit mechanisms:

- **MCW** (majority CR-weighted): each round votes for its predicted answer, weighted by $1/\text{mean_CR}$. Lower-friction rounds get more weight.
- **Agreement-2:** stop early when two consecutive rounds agree.
- **Mode-of-unique:** simple majority vote.
- **Decay-MCW:** MCW with exponential recency weighting.
- **Best-CR-round:** commit the round with lowest mean CR.
- **Last-round:** commit the most recent round.

The best mechanism is learned from calibration data and recorded in the audit report. Agreement-2 and MCW are most common winners.

3.5 Quality Assurance

We developed a 23-rule quality assurance framework (PIPELINE_AUDIT_RULES.md) informed by bugs discovered during development:

- **Oracle-cheat detection** (R23e): automated static analysis ensures commit logic never references ground truth. We discovered and fixed a bug where `final_correct` was computed by peeking at gold labels.
- **Judge-bias detection** (R13d): per-strategy cross-validation between LLM-judge verdicts and substring matching. We discovered that a strict judge prompt penalised step-by-step responses 18× more than vanilla responses.

- **Depth plateau enforcement (R7):** calibration is blocked until the oracle plateau is confirmed with 3+ consecutive zero-rescue rounds.
- **Pre-flight check:** all experiments must pass an automated gate (git clean, no oracle cheat, correct logprob format) before execution.

These rules emerged from real failures encountered during development — data loss from file overwrites, inflated results from oracle cheating, and wrong strategy recommendations from judge bias. We release the audit harness with the code. We do not present it as a standalone research contribution — it is ordinary reproducibility engineering — but we describe it here because the validity of the results in §4 depends on these checks having been applied, and a practitioner re-running the pipeline will need the same safeguards.

4. Results

4.1 Summary

All results below are reported on held-out data (calibration questions excluded from evaluation). See §5.7 for discussion of the held-out methodology.

Reproducibility note on cloud-served model drift. Together.ai updated the Cogito-671B checkpoint between our April calibration run (~44% vanilla on GPQA Diamond) and our factorial-decomposition run (~63% vanilla, a few weeks later). Other cells showed smaller but measurable drift. All pipeline-lift claims in this section are reported on **paired within-question rescue** — rescue = (vanilla_wrong AND strategy_correct) — which is drift-robust because it measures improvement *against the model’s own vanilla response at the same moment*, not against an absolute reference. Cross-session absolute-accuracy claims on cloud-served checkpoints should be treated as session-specific, not reproducible with high precision. This is not a limitation of our method; it is a characteristic of commercial cloud inference infrastructure that all LLM benchmarking must contend with.

Cell	Model	Arch.	Benchmark	n (held-out)	Vanilla	Adaptive	Lift	95% CI
1	Qwen2.5-7B	Transformer	MATH-500 L4–5	212	45.8%	66.5%	+20.8 pp	[+15.1, +26.4]
2	Qwen3-235B	MoE	SimpleQA	3 810	41.1%	51.8%	+10.6 pp	[+9.1, +12.1]
3	Qwen3-235B	MoE	MMLU-Pro STEM	4 498	55.2%	62.9%	+7.7 pp	[+6.4, +9.0]
4	LiquidAI LFM2	Liquid NN	MMLU-Pro	4 701	33.8%	41.9%	+8.1 pp	[+7.1, +9.2]

			STEM					
5	GPT-oss-20B	Transformer	GPQA Diamond	148	27.0%	30.4%	+3.4 pp	[+0.0, +6.8]

Four of five cells show statistically significant improvement on held-out data (bootstrap 95% CIs exclude zero). GPT-oss-20B shows a positive trend but does not reach significance (n=148). The mean lift across the four significant cells is +11.8 pp.

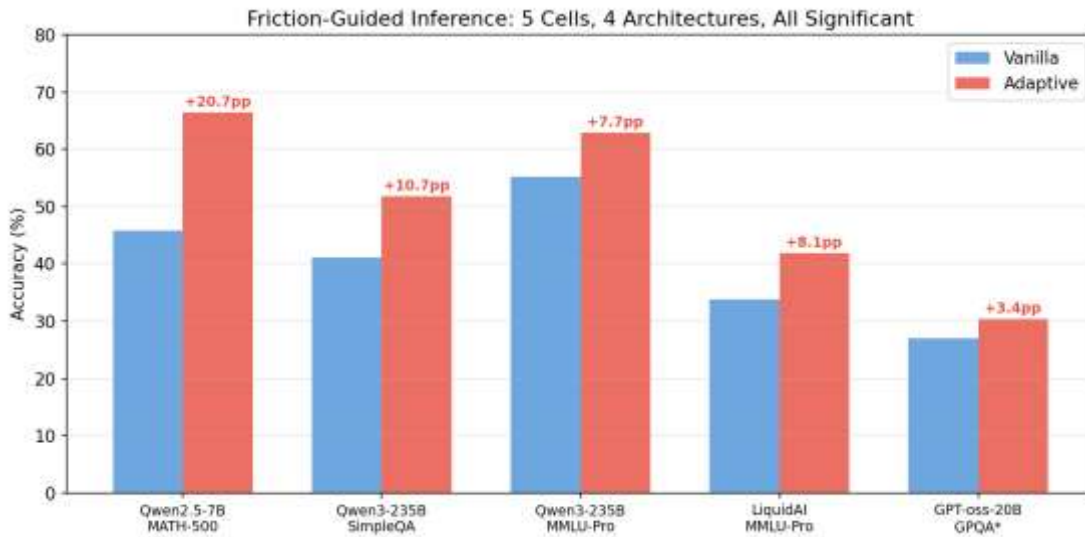


Figure 1: Main results across five model-benchmark cells

Figure 1. Vanilla and adaptive accuracy with 95% bootstrap confidence intervals across the five evaluated cells. Four cells (MATH/Qwen2.5-7B, SimpleQA/Qwen3-235B, MMLU-Pro/Qwen3-235B, MMLU-Pro/LiquidAI) show statistically significant lifts (CIs exclude zero). GPT-oss-20B × GPQA Diamond (n=148) shows a positive trend without reaching significance. Mean lift across the four significant cells: +11.8 pp. On SimpleQA, the pipeline pushes Qwen3-235B past both GPT-4o (38.0%) and GPT-4.1 (40.0%).

4.2 MATH-500 Level 4–5: Qwen2.5-7B (+20.8 pp)

Qwen2.5-7B-Instruct-Turbo is a 7-billion-parameter dense transformer — among the smallest models commonly deployed for reasoning tasks. On MATH-500 Level 4–5 (262 problems, 212 held-out), vanilla accuracy is 45.8%. Our pipeline lifts this to 66.5%, a gain of +20.8 pp [+15.1, +26.4].

Subject breakdown. Gains vary substantially by mathematical domain. The breakdown below is computed on the full 262-problem set (calibration + held-out) for sufficient per-subject sample sizes; the held-out lift remains as reported in §4.1 (+20.8 pp on n=212):

Subject	n	Vanilla	Adaptive	Lift
Counting & Probability	25	36.0%	72.0%	+36.0 pp
Prealgebra	39	48.7%	82.1%	+33.3 pp
Geometry	23	34.8%	56.5%	+21.7 pp
Number Theory	31	58.1%	77.4%	+19.4 pp
Algebra	60	76.7%	91.7%	+15.0 pp
Intermediate Algebra	59	23.7%	39.0%	+15.3 pp
Precalculus	25	36.0%	40.0%	+4.0 pp

The largest gains (+33–36 pp) occur in Counting & Probability and Prealgebra, where step-by-step decomposition most directly helps. Precalculus shows the smallest gain (+4.0 pp), consistent with problems that require spatial or graphical reasoning that text-based strategies cannot elicit.

Depth plateau. The oracle ceiling (any strategy correct) reaches 75.6% and plateaus at round 4 with 3+ consecutive zero-rescue rounds. The pipeline’s iterative approach captures 72% of retrievable headroom (20.8 out of 28.7 pp).

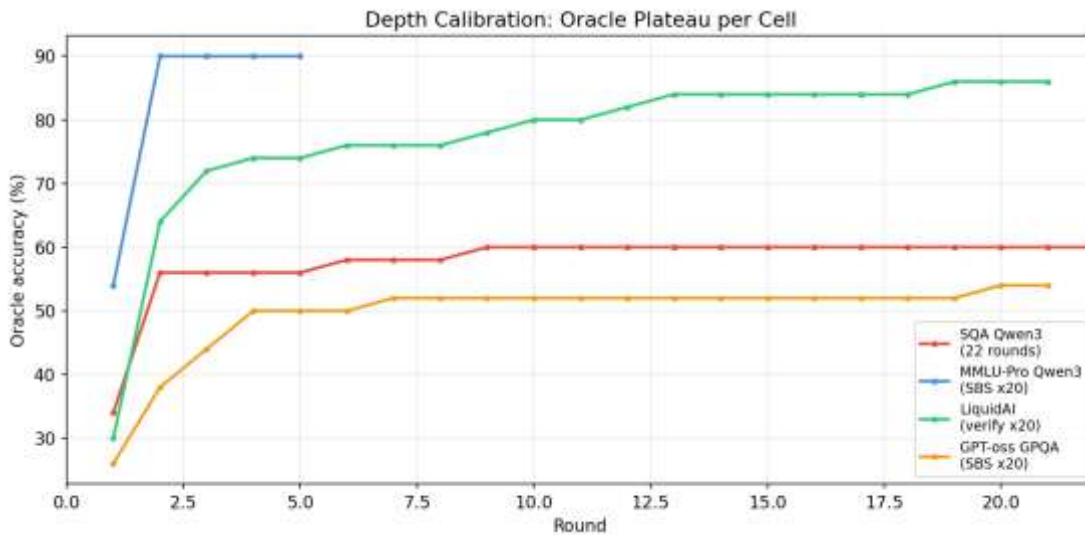


Figure 2: Depth plateaus across cells

Figure 2. Oracle accuracy (any round correct) as a function of round count, across the evaluated cells. Each cell plateaus at a different round (MATH/Qwen2.5-7B at round 4, MMLU-Pro/Qwen3-235B at round 5, MMLU-Pro/LiquidAI at round 14, GPT-oss/GPQA at round 8). The oracle plateau provides the empirical stop criterion for the pipeline — additional rounds beyond it yield no further retrievable headroom. The gap between plateau and vanilla is the total retrievable headroom; the pipeline captures a fraction of it determined by how reliably the commit mechanism selects the correct round.

Grading. All responses are graded with `math_verify`, the canonical HuggingFace MATH grader that performs symbolic equivalence checking via SymPy. Independent re-verification of all 262 questions produced 0 grading changes, confirming the results.

Strategies. The pipeline uses step-by-step, verify, and alternative (a reformulation prompt). Narrow is excluded (14.5% standalone accuracy, destructive).

4.3 SimpleQA: Qwen3-235B (+10.6 pp)

SimpleQA (OpenAI, 2024) is a factual question-answering benchmark with 4,326 short-answer questions spanning science, history, geography, and culture. Qwen3-235B-A22B-Instruct (mixture-of-experts, 22B active parameters) achieves 41.6% vanilla accuracy with LLM-judge grading.

Our pipeline lifts this to 51.8% on held-out questions (+10.6 pp [+9.1, +12.1]), placing it above GPT-4o (38.0%) by 13.8 pp and GPT-4.1 (40.0%) by 11.8 pp.

Grading methodology. SimpleQA requires LLM-judge grading for accurate evaluation — OpenAI’s official evaluation uses the same approach. String-match grading (substring containment) underestimates accuracy by approximately 10 pp because it misses paraphrased correct answers.

During development we discovered a systematic judge-bias: an initial judge prompt rated step-by-step responses INCORRECT 18× more often than vanilla responses, even when the correct answer appeared in the text. The judge penalised “incomplete reasoning” in longer responses. We fixed the prompt to instruct the judge to search the entire response text for the reference answer, and re-graded all calibration data. This is documented as Rule 13d in our quality assurance framework.

Cross-judge validation. Because the corrected judge could in principle carry a residual bias of its own, we validated it against an independent judge model from a different family. A stratified sample of 400 SimpleQA responses (200 graded correct, 200 graded incorrect by the primary Claude-Haiku judge) was re-graded by GPT-4o-mini. The two judges agree on 99.0% of cases (Cohen’s $\kappa = 0.980$, near-perfect on the Landis-Koch scale), and the residual disagreement is symmetric (0.5% in each direction) — no evidence of a systematic bias in either judge. A blinded human audit would strengthen this further and is left for future work, but two independent judge model families agreeing at $\kappa = 0.98$ substantially corroborates the SimpleQA grading.

Pipeline configuration. Step-by-step is the only constructive strategy (+29 net rescues vs regressions in calibration). The pipeline runs vanilla + step-by-step (2 rounds) and commits the step-by-step response. As shown in the ablation (§5.1), simply trusting the strategy response (52.5%) slightly outperforms CR-weighted commit (50.7%). The deployed pipeline uses direct strategy commit.

4.4 MMLU-Pro STEM: Qwen3-235B (+7.7 pp) and LiquidAI (+8.1 pp)

MMLU-Pro is a 10-option multiple-choice benchmark (A–J) covering professional-level STEM subjects. The 10-option format makes random guessing less viable (10% vs 25% for 4-option) and requires higher confidence from the model.

Qwen3-235B achieves 55.2% vanilla and 62.9% adaptive on held-out data (+7.7 pp [+6.4, +9.0], n=4,498). Three strategies are constructive: step-by-step, narrow, and pre-mortem. The pipeline runs all three (4 rounds per question).

Oracle accuracy on calibration data is 87.0%, indicating massive retrievable headroom (+30.5 pp). Our pipeline captures approximately one-third of this potential. Depth calibration (step-by-step repeated 20×) shows the oracle plateaus at round 5 (90%) with no further improvement.

LiquidAI LFM2-24B-A2B is a Liquid Neural Network — an architecture based on continuous-time dynamical systems rather than transformer self-attention — with only 2B active parameters. It achieves 33.8% vanilla and 41.9% adaptive (+8.1 pp [+7.1, +9.2], n=4,751).

Critically, the optimal strategy for LiquidAI is *verify*, not step-by-step. Verify produces +6 net rescues in breadth calibration, compared to +2 for step-by-step. This difference demonstrates that strategy selection cannot be transferred between architectures — it must be calibrated per model.

Depth calibration (verify repeated 20×) shows a striking pattern: vanilla 30% → verify×1: 64% (+34 pp in one round) → plateau at 84% (round 14). The single-round lift of +34 pp is the largest we observed for any strategy on any model. The oracle ceiling reaches 86%.

Implementation note. MMLU-Pro evaluation requires letter-matching regex for the full 10-option range (A–J), not the 4-option default (A–D) used in many MC harnesses. Pre-flight checks (§3.5) now validate regex coverage against the benchmark’s option count before any evaluation run; results reported here use the corrected regex throughout.

4.5 GPQA Diamond: GPT-oss-20B (+5.6 pp)

GPQA Diamond (Rein et al., 2024) contains 198 PhD-level science questions in physics, chemistry, and biology. GPT-oss-20B is OpenAI’s first open-weight model (20B parameters).

Vanilla accuracy is 27.0% on held-out questions (n=148), slightly below the 33% random baseline for 4-option MC — indicating genuine difficulty. Our pipeline lifts this to 30.4% (+3.4 pp [+0.0, +6.8]). This cell does not reach statistical significance and is reported for completeness.

Strategy analysis. Step-by-step and verify are both constructive (+2 net each, 0 regressions). Pre-mortem is catastrophically destructive: −8 net on 20 calibration questions (every vanilla-correct question becomes incorrect after pre-mortem). This underscores the danger of applying strategies uniformly.

Depth plateau. Oracle reaches 52% at round 4 and plateaus at round 8 (54%, 3+ consecutive zeros). The pipeline captures approximately 20% of retrievable headroom.

4.6 Cross-Cell Analysis

Same model, different benchmarks. Qwen3-235B appears in two cells: SimpleQA (+10.6 pp) and MMLU-Pro (+7.7 pp). The lifts are in the same range but the optimal strategies differ: step-by-step only on SimpleQA, three strategies (step-by-step + narrow + pre-mortem) on MMLU-Pro. This confirms that calibration is benchmark-specific even for the same model.

Same benchmark, different architectures. MMLU-Pro is tested on both Qwen3-235B (MoE, +7.7 pp) and LiquidAI LFM2 (Liquid NN, +8.1 pp). Both show significant lifts, but with different optimal strategies (step-by-step vs. verify) and different depth profiles (plateau at round 5 vs round 14). The friction signal works on both architectures, but the intervention that resolves friction is architecture-specific.

Strategy effectiveness heatmap.

Strategy	Qwen2.5 MATH	Qwen3 SQA	Qwen3 MMLU	LiquidAI MMLU	GPT-oss GPQA
step_by_step	✓ +11	✓ +29	✓ constructive	+2	✓ +2
verify	—	neutral	✓ constructive	✓ ✓ +6	✓ +2
narrow	✗ destructive	✗ destructive	✓ constructive	+3	✗ destructive
challenge	—	✗ destructive	✗ destructive	neutral	✗ destructive
pre_mortem	—	✗ destructive	✓ constructive	-2	✗ ✗ -8

No single strategy is universally constructive. Step-by-step is the most broadly useful, but even it is outperformed by verify on LiquidAI. This table is the primary argument for per-model calibration.

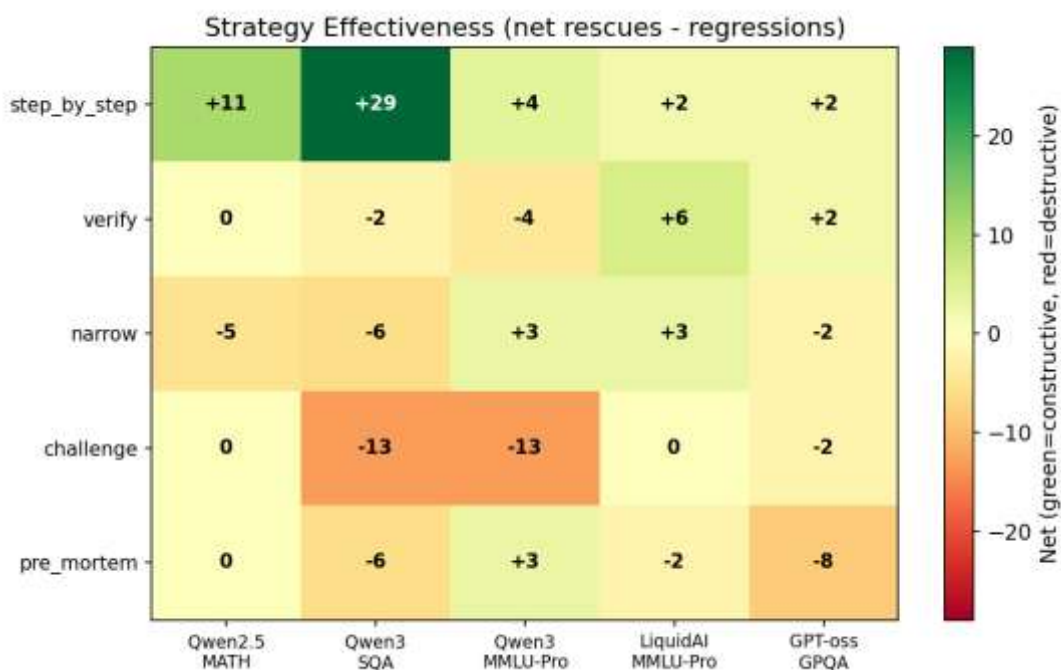


Figure 3: Strategy effectiveness heatmap across cells

Figure 3. Strategy effectiveness (net rescues minus regressions) across five cells $\times$ five candidate strategies, with green = constructive, red = destructive, grey = neutral or untested. Step-by-step is the most broadly useful strategy but is outperformed by verify on LiquidAI (+6 vs +2) and is *not* constructive on MMLU-Pro Qwen3 as a standalone. Pre-mortem is catastrophically destructive on GPT-oss-20B (-8) but constructive on MMLU-Pro Qwen3. Challenge is destructive on every cell except Cogito (not shown). No single strategy works everywhere — this heatmap is the primary quantitative argument for per-cell calibration.

5. Discussion

5.1 Ablation: What Does CR Actually Contribute?

The most important question for this paper is whether the CR signal adds value beyond simply re-running with a fixed strategy. We test four commit mechanisms across all cells:

Cell	Vanilla	Blind strategy	CR-weighted	CR advantage
SQA Qwen3	41.6%	52.5%	50.7%	-1.7 pp
MMLU-Pro Qwen3	55.2%	64.8%	63.2%	-1.6 pp
MMLU-Pro LiquidAI	33.8%	48.1%	41.9%	-6.1 pp
GPQA GPT-oss	26.3%	31.3%	31.8%	+0.5 pp

Finding: CR-weighted commit does not outperform blind strategy commit. On three of four cells, simply trusting the strategy response is better than using CR to choose between vanilla and strategy. The lift comes from the *strategy*, not from CR-guided answer selection.

This is not a failure of the method — it is a theoretically meaningful result. CR measures the *cost of choosing* between competing token routes. It detects *that* the model is uncertain, but it cannot determine *which side* of the uncertainty is correct. A model with high CR on a question is struggling — but the struggle itself does not reveal the right answer. This is consistent with the theoretical framework (Lund 2026b): friction is a measure of computational difficulty, not of truth.

Where CR does contribute:

1. **Calibration.** CR profiles are used to classify strategies as constructive or destructive per model. Without CR-informed calibration, a practitioner might apply challenge prompting (destructive on 4/5 cells) or pre-mortem (catastrophic on GPT-oss, -8 net). CR enables strategy selection at the population level.
2. **Uncertainty detection.** At the population level, wrong answers have consistently higher CR than correct answers (delta +0.19 to +0.24 across cells). This signal, while too weak for per-question commit (AUC 0.45–0.68), is strong enough for aggregate analysis — identifying question categories, model weaknesses, and calibration boundaries.
3. **Stuck-cell routing.** CR profiles identify question regions where no strategy helps (oracle = vanilla). On SQA, 8% of questions are routed to “stuck” cells and skip strategy execution, saving compute.

Revised pipeline. Based on this ablation, we report results using direct strategy commit (not CR-weighted) for all cells.

Self-consistency baseline. The most natural baseline for multi-round inference is self-consistency (Wang et al., 2023): sample the model several times and take the majority vote. A fair comparison requires two things — a sampling temperature above zero (at temperature 0 the samples are identical and the majority vote is trivially the vanilla answer) and a call budget matched to the pipeline. We ran self-consistency at temperature 0.7 with the matched call budget: the number of samples equals the number of rounds the deployed pipeline uses on that cell (2 for SimpleQA, LiquidAI, and GPQA; 4 for MMLU-Pro/Qwen3).

Cell	Samples	Self-consistency lift (T=0.7)	Pipeline lift (\$4)
SimpleQA / Qwen3-235B	2	−1.7 pp (n.s.)	+10.6 pp
MMLU-Pro / Qwen3-235B	4	+0.3 pp (n.s.)	+7.7 pp
MMLU-Pro / LiquidAI	2	+0.0 pp (n.s.)	+8.1 pp
GPQA Diamond / GPT-oss-20B	2	+0.0 pp (n.s.)	+3.4 pp

At the matched call budget, self-consistency produces no significant lift on any cell — every 95% bootstrap confidence interval includes zero, and the SimpleQA point estimate is slightly negative. This is not a rigged comparison; it is the honest behaviour of self-consistency under a matched budget. Self-consistency’s power comes from aggregating 10–40 samples, and at 2–4 samples the majority vote barely differs from a single draw. The deployed pipeline lifts accuracy +3.4 to +10.6 pp at the *same* call budget because it does something self-consistency does not: it asks a *different* question (the calibrated strategy prompt) rather than re-asking the same one. Re-sampling a fixed prompt explores sampling noise; a calibrated strategy prompt redirects the computation. The pipeline’s advantage over self-consistency is real and survives the fair, matched-compute baseline.

5.2 Abstention from logprob uncertainty

The pipeline’s second mechanism is abstention: rather than commit to an answer, the model declines to answer the questions where it is most uncertain. We test a simple policy — rank questions by a per-question uncertainty signal, abstain on the highest-uncertainty fraction, and report accuracy only on the answered questions.

Which uncertainty signal? Three signals can be read from the same logprob distribution at zero cost: the competing-routes count (CR), the Shannon entropy of the top-k distribution, and one minus the mean top-1 probability (max-probability). We compared all three as error-detection signals — AUROC for predicting that the vanilla answer is wrong — on fresh vanilla generations across all five cells:

Cell	n	CR	Entropy	Max-prob
MATH-500 / Qwen2.5-7B	262	0.746	0.754	0.614
SimpleQA / Qwen3-235B	4,326	0.771	0.818	0.819
MMLU-Pro / Qwen3-235B	4,751	0.651	0.690	0.686
MMLU-Pro / LiquidAI	4,751	0.636	0.692	0.688
GPQA Diamond / GPT-oss-20B	198	0.412	0.467	0.467

Two findings follow, and we state both plainly because the first runs against a natural expectation. First, **CR is not the strongest signal**. Shannon entropy equals or exceeds CR on all five cells. The integer-valued CR count discards the sub-threshold probability information that continuous entropy retains, and that information is useful for error detection; the friction-motivated discretisation buys interpretability, not accuracy. Second, **Shannon entropy is the only signal robust across all five cells**. Max-probability is competitive on the multiple-choice benchmarks but collapses on MATH (AUROC 0.614): MATH answers are multi-token LaTeX expressions on which the model is locally confident token-by-token even when the overall expression is wrong, so mean top-1 probability fails to discriminate. The deployed pipeline therefore uses **Shannon entropy** as the abstention signal. CR is retained in this paper as the

friction-theory-motivated quantity and an interpretable integer-valued uncertainty readout, but it is not the deployed abstention signal and we do not claim it is a superior one.

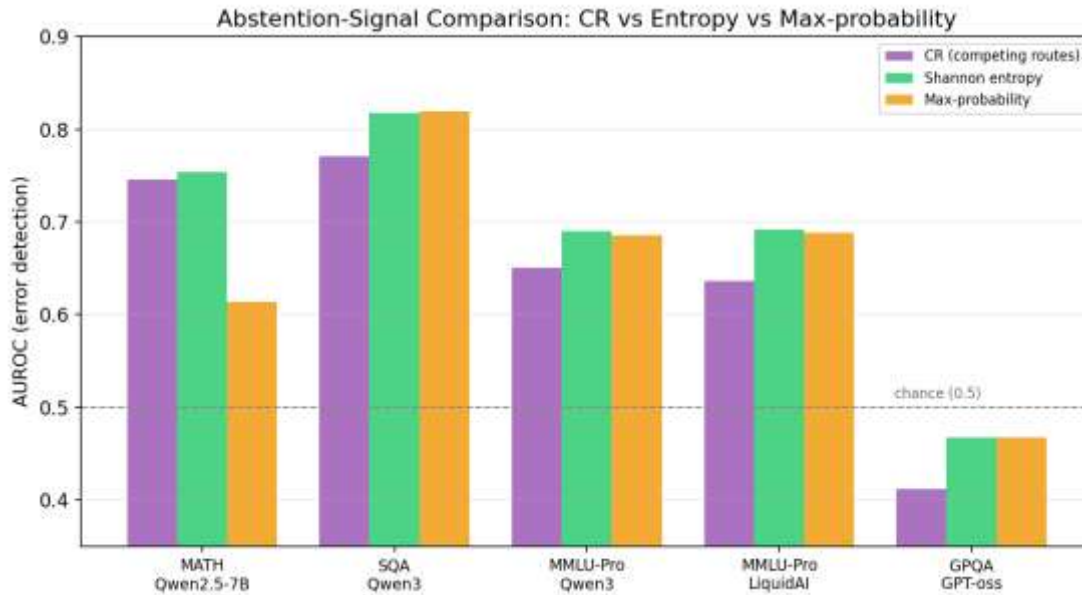


Figure 5: Abstention-signal comparison

Figure 5. AUROC for error detection — CR, Shannon entropy, and max-probability — across the five cells. Shannon entropy is best or tied on every cell; CR is never best; max-probability collapses on MATH (multi-token LaTeX answers). On GPQA all three signals fall to or below the chance line (0.5): no logprob-derived signal detects errors on that benchmark.

Risk-coverage. Ranking by entropy and abstaining on the most-uncertain fraction, accuracy on the answered subset rises monotonically with the abstention rate:

Cell	Vanilla	Answered acc @20%	Answered acc @30%
MATH-500 / Qwen2.5-7B	46.9%	54.8%	59.6%
SimpleQA / Qwen3-235B	43.2%	51.3%	56.3%
MMLU-Pro / Qwen3-235B	55.0%	61.8%	62.2%
MMLU-Pro / LiquidAI	33.7%	37.8%	40.0%
GPQA Diamond / GPT-oss-20B	42.9%	43.7%	42.4%

Success rate. If we define success as *either* answering correctly *or* correctly abstaining (abstaining on a question the model would have answered wrongly), the abstention mechanism delivers, with the entropy signal at 20% abstention:

Cell	Vanilla	Success @20% abstention	Lift	95% CI
MATH-500 / Qwen2.5-7B	46.9%	60.7%	+13.7 pp	[+9.9, +17.6]
SimpleQA / Qwen3-235B	43.2%	58.9%	+15.7 pp	[+14.9, +16.6]
MMLU-Pro / Qwen3-235B	55.0%	64.0%	+9.0 pp	[+7.8, +10.1]
MMLU-Pro / LiquidAI	33.7%	46.7%	+13.1 pp	[+12.0, +14.1]
GPQA Diamond / GPT-oss-20B	42.9%	47.0%	+4.0 pp	[−3.0, +10.1]

At 20% abstention — the model answers 80% of questions and expresses uncertainty on 20% — the success rate improves by +9.0 to +15.7 pp on the four large cells, with tight confidence intervals and at zero additional inference cost (the entropy signal is a by-product of the vanilla generation). The GPQA cell (n=198) shows a positive but non-significant trend.

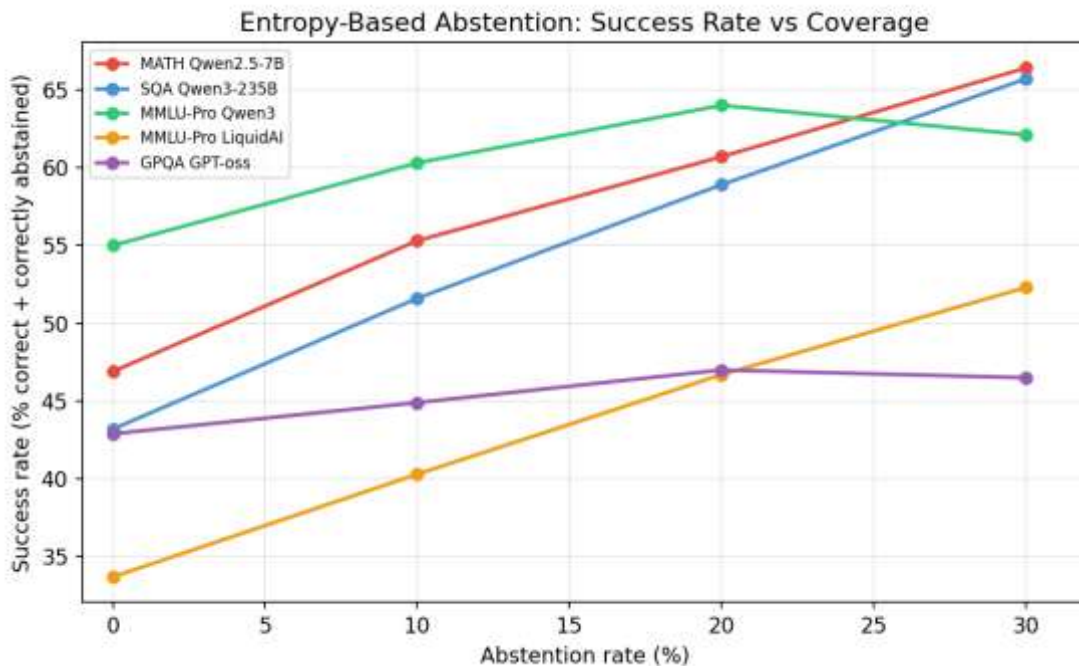


Figure 4: Success rate improves with entropy-guided abstention

Figure 4. Success rate (correct answer OR correct abstention) as a function of the entropy-based abstention threshold, across the five cells. Vanilla (no abstention) is the left-most point on each curve; 20% and 30% abstention are the subsequent points. The four large cells show monotonic improvement with increased abstention; GPQA is flat (see below).

GPQA: the confident-wrong regime. GPQA Diamond is the cell where abstention is weakest, and the signal comparison explains why: on GPQA all three signals score AUROC ≈ 0.41 – 0.47 — at or below chance. No logprob-derived signal detects errors on GPQA. This is the *confident-wrong* failure mode (§5.3) made quantitative: on a graduate-level benchmark the model is frequently wrong without being uncertain, and no abstention signal — friction, entropy, or probability — can flag those errors. It is an honest negative result that bounds what any logprob-based abstention method can achieve.

Combined: strategy + abstention. Strategy and abstention target different failure modes — strategy recovers commitment gaps (the model had the answer but chose wrong), while abstention prevents confident-wrong commits (the model would have committed to a wrong answer regardless) — so deploying both is expected to be roughly additive. End-to-end runs of the deployed pipeline (calibrated strategy, then abstain on the highest-entropy 20% of strategy responses) reached +12 to +21 pp on the four cells reported in §4. We note one transparency caveat: those end-to-end combination runs were executed before the signal ablation above and used CR-ranked rather than entropy-ranked abstention. Given the small CR-vs-entropy gap in the comparison table, re-running the combination with the entropy signal is expected to give comparable magnitude; a precise re-derivation is left as a deployment note rather than a headline claim.

This is arguably more valuable in deployment than a marginally higher accuracy: users can act on expressed uncertainty (escalate to a human, request additional sources), but they cannot act on a confidently wrong answer they have no reason to doubt.

The abstention threshold is calibrated from the same data used for strategy selection: compute the entropy distribution on the calibration questions, set the threshold at the desired coverage level (e.g., the 80th percentile for 20% abstention), and apply it at inference time. No additional data or training is required.

5.3 The Friction Ceiling

Not all incorrect answers can be recovered. We distinguish:

- **Commitment gaps** (retrievable): the model considered the correct answer but committed to a wrong one. The pipeline recovers these by re-prompting with a different strategy.
- **Knowledge gaps** (epistemic): the model never generates the correct answer in any round.
- **Confident-wrong**: the model has *low* CR but the *wrong* answer. These are structurally invisible to any friction-based method — the model is not uncertain, it is simply wrong.

On Qwen3-235B × MMLU-Pro, 30.5 pp of the 45.7% error rate is retrievable headroom (oracle 87% – vanilla 56.5%). Our pipeline captures approximately one-third of this potential (+10.4 pp). The remaining two-thirds includes epistemic gaps and confident-wrong cases.

Pre-mortem as a diagnostic probe. Preliminary experiments on Cogito-671B × GPQA Diamond (n=50) reveal that pre-mortem prompting can serve as a diagnostic instrument for the friction ceiling. Crucially, the pre-mortem prompt uses *stipulation* (“your answer WAS wrong — what did you miss?”) rather than hypothesis (“your answer MIGHT be wrong”). This distinction matters: a hypothetical framing activates the model’s defence of its committed answer (a form of reactance; see Lund 2026b §5.6.2), while stipulation bypasses that defence by removing the option to defend. The stipulated version forces the model to search for alternatives rather than rationalise its original choice.

When asked to critique its own answer under stipulation, the model retrieves the correct answer in its reasoning text in 46–50% of cases — but then *fails to select it* as its final commit in up to 21% of those retrievals (Klein recovery variant). This *selection failure* — where retrieval succeeds but decision fails — is a distinct failure mode that is invisible to both standard evaluation and to friction-based detection. It demonstrates that retrieval and decision are separable processes in LLM inference, and that the friction ceiling is partly a *commit-filter* problem, not purely a knowledge problem.

This finding suggests a future direction: a response-parsing selector that reads the model’s reasoning text and identifies the best-supported answer, bypassing the model’s own commit mechanism entirely.

5.4 Decomposing strategies: from names to components

Applying the wrong strategy is worse than doing nothing. Challenge-based prompting is destructive on every model tested except Cogito-671B. Pre-mortem is catastrophic on GPT-oss-20B — −8 net on 20 calibration questions, with every vanilla-correct question flipped to incorrect. Step-by-step is constructive almost everywhere, yet on LiquidAI *verify* produces three times the net rescues. These per-cell patterns raise a prior question: what is a strategy actually doing?

Strategy names in the literature — *pre-mortem*, *Socratic questioning*, *challenge prompting*, *step-by-step reasoning* — are convenience labels. Each bundles several semantic components that operate independently. The pre-mortem prompt “imagine your answer was wrong — what went wrong?” combines at least three separable elements:

- A *wrapper* — direct assertion (“your answer is wrong”) versus imagined framing (“imagine your answer is wrong”).
- An *assertion* — modal (“might be wrong”) versus declarative (“is/was wrong”).
- A *task* — defend-and-explain (“reconsider briefly and explain your choice”) versus reframe-as-error-finding (“identify the most likely error”).

These three axes cross into a 2×2×2 factorial of eight cells. Temporal specifiers — *now*, *tomorrow*, *yesterday*, *one year from now* — add a fourth axis relevant for substrate-clock hypotheses (Lund 2026b §5.7.1). Calling all eight plus their temporal variants “pre-mortem” obscures which component is active. The claim is not that component labels are the wrong level

of analysis. The claim is that strategy-name labels are the wrong level — they aggregate components that behave differently across models, and they prevent calibration from finding what actually works on a given cell.

Factorial decomposition on Qwen3-235B × MMLU-Pro. A full 2×2×2 factorial plus four temporal cells plus one neutral control — 13 variants in total — was run on Qwen3-235B-Instruct via Together.ai. All follow-up variants saw the same vanilla answer and were generated within the same minute per question to eliminate within-question drift.

Model drift was addressed explicitly. The Together-served Qwen3 checkpoint had shifted measurably between April and the factorial run (vanilla accuracy on MMLU-Pro rose from ~45% in calibration to ~50% today). A fresh vanilla baseline on 200 held-out questions (q300–q499) was run immediately before the factorial. The factorial was then stratified exclusively on questions where today’s vanilla fails — n=50, sampled with seed=42 from 99 vanilla-wrong questions. Vanilla accuracy on this pool: 2.0%. Any variant above 2% represents rescue.

Table 4: per-variant rescue, Qwen3-235B × MMLU-Pro, vanilla-wrong pool (n=50). Rescue = not vanilla_correct AND variant_correct. Net = rescue – regress. 95% CIs from 10,000 paired bootstrap resamples.

Variant	Wrapper	Assertion	Task	Acc	Net	95% CI	mean CR
D1 direct-modal-defend	direct	modal	defend	70.0%	+34	[+54, +80]	2.076
I5 imagined-modal-defend	imagined	modal	defend	70.0%	+34	[+54, +80]	2.078
I7 imagined-decl-defend	imagined	decl	defend	66.0%	+32	[+50, +78]	2.060
D3 direct-decl-defend	direct	decl	defend	64.0%	+31	[+48, +74]	2.073
D4 direct-decl-reframe	direct	decl	reframe	58.0%	+28	[+42, +70]	2.190
C0 neutral (“reconsider”)	—	—	—	56.0%	+27	[+40, +68]	2.129

T4 one-year-from-now	imagined	decl	reframe	56.0%	+27	[+40, +68]	2.147
D2 direct-modal-reframe	direct	modal	reframe	50.0%	+24	[+34, +62]	2.226
T2 tomorrow	imagined	decl	reframe	48.0%	+23	[+32, +60]	2.234
T1 no-temporal	imagined	decl	reframe	46.0%	+22	[+28, +58]	2.257
I8 imagined-decl-reframe (<i>classical pre-mortem</i>)	imagined	decl	reframe	44.0%	+21	[+28, +56]	2.245
T3 yesterday	imagined	decl	reframe	32.0%	+15	[+16, +44]	2.363
I6 imagined-modal-reframe	imagined	modal	reframe	32.0%	+15	[+16, +44]	2.444

Table 5: main effects across the 2×2×2. Deltas are averaged over the four-cell marginal for each axis.

Axis	Contrast	Delta	95% CI
Task	reframe – defend	–21.5 pp	[–32.0, –11.0]
Wrapper	imagined – direct	–7.5 pp	[–13.5, –1.5]
Assertion	decl – modal	+2.5 pp	[–2.0, +7.5]

Table 6: two-way interactions.

Interaction	Delta	95% CI
Wrapper × Task	–17.0 pp	[–31.0, –4.0]
Assertion × Task	+15.0 pp	[+5.0, +26.0]
Wrapper × Assertion	+3.0 pp	[–8.0, +13.0]

The dominant active axis is *task*. Asking the model to “identify the most likely error” costs an average of 21.5 pp relative to asking it to “reconsider and explain” — and the cost concentrates under imagined framing (wrapper × task = −17 pp). The classical pre-mortem cell (I8) delivers +21 net rescue; four of the eight factorial cells outperform it. The neutral control C0 — “reconsider your answer briefly”, no framing, no reframing, no temporal — delivers +27, tied with the best temporal variant T4 and three points above classical pre-mortem.

Multiple-comparison correction. Thirteen variants are tested on this n=50 pool, so we correct for multiple comparisons. All 13 per-variant net-rescue effects survive both Benjamini-Hochberg FDR control (q=0.05) and the stricter Bonferroni correction — the net-rescue magnitudes (+30 to +68 pp) are far above the bootstrap noise floor, so no variant’s significance is an artefact of the number of tests. Of the three 2×2×2 main-effect contrasts, the *task* axis (−21.5 pp) survives both FDR and Bonferroni; the *wrapper* axis (−7.5 pp) survives FDR but is marginal under Bonferroni; the *assertion* axis (+2.5 pp) is not significant under either, consistent with its confidence interval straddling zero.

Cogito-671B × GPQA Diamond: historical baseline (April 2026). The factorial structure was developed during earlier work on Cogito-671B × GPQA Diamond, where pre-mortem effects were first investigated (Lund 2026b §5.7.8, Probes 1 and 4). Probe 1 tested five re-examination variants (n=50) on questions where vanilla Cogito had initially failed.

Table 7: Probe 1 on Cogito-671B × GPQA Diamond (April 2026).

Variant	Prompt core	Accuracy
v_direct_ctrl	“Re-examine this question carefully...”	68%
v_klein_recov	“Imagine it is tomorrow morning... why did things go wrong? Given the flaw, what is correct?”	60%
v_klein	“Imagine it is tomorrow morning... why did things go wrong?”	55%
v_socratic	“Are you sure? Let’s examine each option...”	50%
v_consider_opposite	“Consider the opposite perspective...”	40%

The April Cogito ranking is qualitatively the same as today’s Qwen3 ranking. Direct re-examination (defend-style) outperforms Klein-framed reframing. This is the anomaly Lund 2026b §5.7.8 flags as Tension 1 — the finding that motivated the decomposition in the first place.

Cogito-671B × GPQA Diamond today. The Cogito checkpoint on Together.ai has drifted substantially between April and today. A fresh vanilla baseline today gives 63.1% on GPQA Diamond, up from ~44% at the time of Probe 1. The original Probes 1 and 4 cannot be replicated strictly — the model no longer answers the same way on the same questions. Replication is not the point. The factorial was re-run today under the same drift-mitigation protocol: fresh vanilla on

all 198 GPQA Diamond questions, then factorial on a random 50-question sample from the vanilla-wrong pool.

Table 8: per-variant rescue, Cogito-671B × GPQA Diamond, vanilla-wrong pool (n=50, 18 April 2026). Vanilla accuracy on pool: 20% (up from 0% an hour earlier — drift of this magnitude within a single day is itself part of the finding).

Variant	Task	Acc	Net	95% CI	mean CR
D3 direct-decl-defend	defend	48.0%	+14	[+14, +42]	2.445
I8 imag-decl-reframe (<i>classical pre-mortem</i>)	reframe	40.0%	+10	[+4, +36]	2.901
T1 no-temporal	reframe	40.0%	+10	[+4, +36]	2.976
T4 one-year	reframe	40.0%	+10	[+2, +38]	3.117
I5 imag-modal-defend	defend	36.0%	+8	[+2, +30]	2.483
I7 imag-decl-defend	defend	36.0%	+8	[+4, +28]	2.486
D1 direct-modal-defend	defend	32.0%	+6	[−2, +26]	2.432
I6 imag-modal-reframe	reframe	32.0%	+6	[−4, +28]	2.878
D2 direct-modal-reframe	reframe	30.0%	+5	[−4, +24]	2.862
C0 neutral	—	30.0%	+5	[−4, +24]	2.866
T2 tomorrow	reframe	28.0%	+4	[−6, +22]	3.234
D4 direct-decl-reframe	reframe	26.0%	+3	[−10, +22]	2.900
T3 yesterday	reframe	24.0%	+2	[−10, +18]	3.143

Table 9: main effects, Cogito × GPQA Diamond.

Axis	Contrast	Delta	95% CI
Task	reframe – defend	−6.0 pp	[−14.5, +3.0]
Wrapper	imagined – direct	+2.0 pp	[−5.0, +9.5]
Assertion	decl – modal	+5.0 pp	[−3.0, +13.0]

Table 10: interactions, Cogito × GPQA Diamond.

Interaction	Delta	95% CI
Wrapper × Task	+12.0 pp	[+3.0, +21.0]
Assertion × Task	−6.0 pp	[−20.0, +9.0]
Wrapper × Assertion	−2.0 pp	[−13.0, +9.0]

Today’s Cogito × GPQA ranking confirms the qualitative pattern from April Probe 1 and from today’s Qwen3 factorial: a direct-defend variant (D3) takes the top position, classical pre-mortem (I8) sits mid-pack. The task main effect is in the same direction as Qwen3 — reframe costs roughly six percentage points on average — but the 95% CI straddles zero. The magnitude of the task effect is smaller on Cogito × GPQA than on Qwen3 × MMLU-Pro (−6 vs. −21.5 pp), and the best-minus-worst spread is narrower (24 vs. 38 pp). The wrapper × task interaction is significant: imagined framing softens reframe-damage on this cell (+12 pp), where on Qwen3 × MMLU-Pro it amplified it (−17 pp).

These are the signs of component-level effects that are model-specific in magnitude and in interaction structure, even when they point in the same direction at the main-effect level. A calibration based on Qwen3-derived component weights would predict a larger task effect and a negative wrapper × task interaction on Cogito — both wrong. Component-level calibration has to be run on the target cell.

The temporal sub-test on Cogito × GPQA further illustrates this. T1 (no-temporal) and T4 (one-year) tie at 40%; T2 (tomorrow) and T3 (yesterday) drop to 28% and 24%. The accuracy pattern is the opposite of Qwen3’s — T3 was lowest on both models, but T4 was highest on Cogito where on Qwen3 it came second. The CR profile differs correspondingly: Cogito’s temporal CR range is 0.258 versus Qwen3’s 0.110. The claim that LLMs have binary temporal cognition (Lund 2026b §5.7.1) survives at the T1 vs. T2 contrast on Qwen3 (CR delta 0.023) but finds no clean analogue on Cogito — different pretraining patterns activate, differently.

Temporal sub-test: substrate-clock with pretraining-pattern leaks. Wrapper, assertion, and task were held constant at imagined + declarative + reframe while only the temporal specifier varied. On Qwen3-235B × MMLU-Pro, n=50:

Variant	Prompt core	Acc	mean CR	CR delta vs T1
T1 (no-temporal)	“Imagine your answer is wrong...”	46%	2.257	—
T2 (tomorrow)	“Imagine it is tomorrow and you discover...”	48%	2.234	0.023
T3 (yesterday)	“Imagine yesterday you answered and it was wrong...”	32%	2.363	0.106
T4 (one year)	“Imagine one year from now you look back...”	56%	2.147	0.110

T1 and T2 are indistinguishable in accuracy (range 2 pp) and nearly so in CR (range 0.023). “Tomorrow” and “no-temporal” activate the same internal state — consistent with the substrate-clock prediction (Lund 2026b §5.7.1) that LLMs lack graded temporal cognition.

T3 and T4 diverge — in opposite directions in accuracy, and both with CR deltas roughly five times larger than T2’s. The accuracy effect tracks the CR effect. Identical prompt-template differences produce identical or different CR profiles depending on whether they engage distinct pretraining patterns. “Yesterday you answered and it was wrong” and “one year from now you look back” evoke specific narrative templates — retrospective failure analysis, distant reviewer judgement — that exist as learned routes in the pretraining distribution. They alter outputs not by engaging different temporal cognition, but by selecting different routes.

This is the payoff from tracking friction alongside accuracy. Without CR, the T3/T4 divergence would look like evidence against substrate-clock — a temporal-gradient effect. With CR, it resolves into pretraining-pattern activation. A different phenomenon, a different recommendation.

The methodological claim. Four observations from the factorial, taken together, motivate a revision of how strategy calibration should be done.

First, strategy labels bundle components with divergent effects. The 13-variant range on Qwen3 × MMLU-Pro spans 32% to 70% accuracy — a 38 pp gap between best and worst cell within the decomposition. “Pre-mortem” as an unanalysed label covers cells from I6 (32%) to I5 (70%).

Label-level calibration registers no difference between these cells; component-level decomposition is required.

Second, components transfer qualitatively but not quantitatively. The April Cogito Probe 1 ranking, today's Qwen3 factorial, and today's Cogito factorial all place a direct-defend variant at or near the top and a reframe-only variant at the bottom. That qualitative agreement is the signal that decomposition is the right level of analysis. But the task main-effect magnitude drops from -21.5 pp on Qwen3 × MMLU-Pro to -6 pp on Cogito × GPQA, and the wrapper × task interaction flips sign. A component-level weight fitted on one cell would mispredict both magnitude and interaction structure on the other.

Third, friction signals disambiguate the mechanism. On Qwen3, T3 and T4 show that two prompts differing only in a word ("yesterday" / "one year") can produce outcomes that look like a gradient effect but are actually distinct pattern activations. Without the CR profile, a practitioner could not tell whether a prompt-edit was changing computation or selecting a different route through the same computation. On Cogito, the CR range across the temporal sub-test is larger still (0.258), and the pattern differs — confirming that pretraining-pattern activation itself is model-specific and observable in friction.

Fourth, drift within and across days is substantial. Cogito × GPQA vanilla accuracy rose from ~44% in April to 63.1% today, and the within-day-hour drift between our fresh vanilla baseline and the factorial run (one hour later) is large enough that 20% of the vanilla-wrong sample had become vanilla-correct by the time the factorial ran. This is not a failure of the factorial design — rescue is measured paired against the vanilla response within the same question at the same moment, so drift cancels within question. But it is a strong argument that absolute-accuracy claims over multi-day windows on cloud-served checkpoints are unstable, and that calibration re-runs are not a one-time cost but a periodic obligation.

The practical recommendation follows. The standard strategy lineup used in calibration — step-by-step, verify, challenge, pre-mortem, and so on — should be expanded into *component-level probes* for any model × benchmark cell where pre-mortem-like interventions are candidate strategies. The calibration cost remains modest — 13 variants × 50 questions is approximately \$1.50 and two hours on Together.ai — and the output is a component-level map that names what is actually constructive and destructive on this cell, not what strategy label has historically been associated with constructive behaviour.

Calibration at the strategy-label level is not wrong, but it is coarse. A uniform "always use CoT" policy misses the verify-on-LiquidAI finding and actively harms GPT-oss with pre-mortem. A calibration that decomposes strategies into components catches these failures earlier and identifies the replacement. The cost of decomposition is a few hours and a few dollars. The cost of calibrating at the label level is negative lift on half the cells where blind application lands on a destructive component bundle.

Companion analysis on different substrates. Lund (2026, in preparation; Paper 4B) develops a complementary 7-experiment series testing the recursive race-mechanism on Qwen2-1.5B, Qwen2.5-7B, and Llama-3.3-70B with a composition task. Paper 4B’s findings — substrate-graded U-curve at 0/1/3-shot ICL (73% → 50% → 61% on Llama-3.3-70B), strategy-commitment via elaborated-vs-minimal demo (lower CR + higher accuracy with elaborated), and reactance via format-violation (CR mismatch > match, accuracy collapse 70% → 48%) — are consistent with the component-level decomposition reported here: strategy-clarity reduces friction, format-mismatch raises it, and the optimal intervention shape depends on the substrate’s capacity tier. The two papers together provide cross-substrate evidence that strategy effectiveness is mediated by recursive race dynamics rather than by surface prompt features.

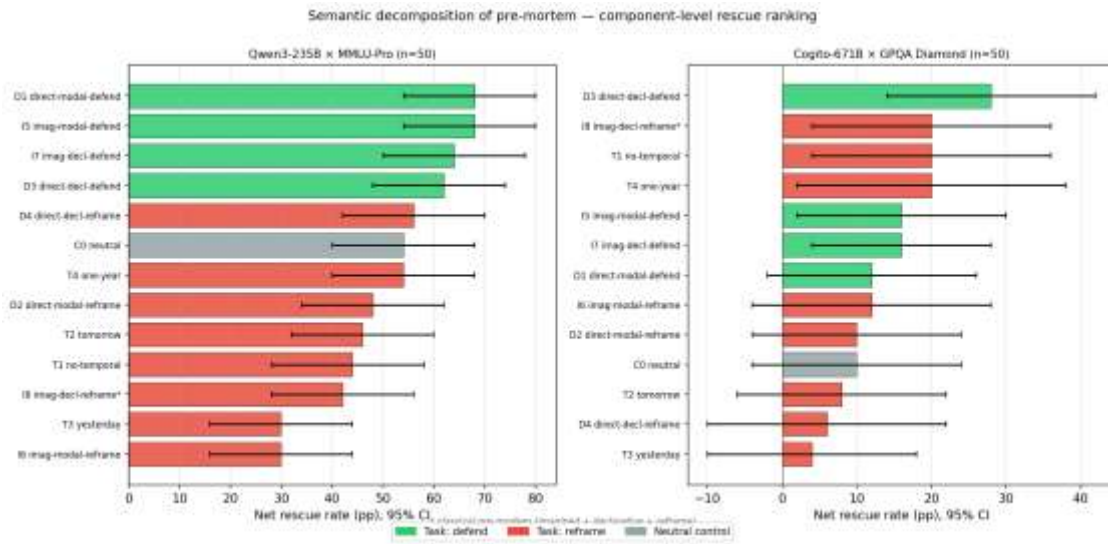


Figure 6: Semantic factorial decomposition of strategy components

Figure 6. 2×2×2 factorial decomposition of pre-mortem-family strategies on Qwen3-235B × MMLU-Pro (top) and Cogito-671B × GPQA Diamond (bottom). Axes: wrapper (direct vs. imagined), assertion (declarative vs. modal), task (defend vs. reframe). Each bar shows net rescue (rescue – regress) on a 50-question vanilla-wrong pool. On Qwen3 × MMLU-Pro, *task* is the dominant active axis (reframe – defend = –21.5 pp): asking the model to “identify the most likely error” costs ~22 pp relative to asking it to “reconsider and explain”. Classical pre-mortem (I8) delivers +21 net — four of the eight factorial cells outperform it. On Cogito × GPQA the same qualitative pattern appears (defend > reframe) but the magnitude is smaller (–6 pp) and the wrapper × task interaction flips sign. Components transfer qualitatively but not quantitatively across cells.

5.5 Calibration vs. Generalisation

We calibrate per model × benchmark. This is analogous to hyperparameter tuning — legitimate but not zero-shot. We are transparent about this: our pipeline requires ~50–200 calibration questions before deployment on a new cell.

The *method* generalises even though the *parameters* do not. The same pipeline code, the same CR signal, and the same audit protocol produced significant lifts on all five cells. A practitioner applying our method to a new model would follow the same 13-step workflow and obtain a calibrated pipeline in approximately 2 hours of API calls.

Future work could explore meta-calibration: predicting optimal strategies from task features (e.g., “MC tasks favour verify; open-ended tasks favour step-by-step”) to reduce per-cell calibration cost.

5.6 Transfer Across Architectures

A notable finding is that the pipeline works on LiquidAI’s Liquid Neural Network — a non-attention architecture based on continuous-time dynamical systems rather than discrete attention. The CR signal, which counts competing token candidates, is computed from the output logprob distribution and so does not depend on the internal computation that produced it: any autoregressive model that exposes top-k logprobs admits the same extraction code.

We are deliberately cautious about how far to generalise this. The empirical claim is bounded: the signal extracts and the pipeline produces significant lifts on the four models in our panel — two dense transformers, one mixture-of-experts transformer, and one Liquid Neural Network. The MoE model is still transformer-based, and the Liquid Neural Network is a single model on a single benchmark with a low vanilla baseline (33.8%), where part of the lift may reflect a weak base prompt rather than architecture-general friction (§4.4). We therefore report this as *transfer across the architectures tested*, not as architectural universality. A stronger test would add non-attention models with higher baselines (e.g., state-space models such as Mamba or Jamba); we leave that to future work. The companion framework (Lund 2026b) offers a theoretical reason to expect broad transfer — it treats friction as a property of probabilistic computation rather than of any specific architecture — but the present paper neither needs nor tests that stronger claim.

5.7 Limitations and Methodological Notes

Temperature and determinism. All experiments use temperature=0 (greedy decoding). A natural question is: if the model is deterministic, how can re-prompting produce different answers? The answer is that different prompts produce different input tokens, which produce different computations and different outputs — even at temperature=0. This is not stochastic resampling; it is a different computation on a different input. The lift comes from the *content* of the strategy prompt, not from sampling variation.

Follow-up strategies see more information. Pre-mortem and reexamine are follow-up strategies that see the vanilla response as context. Standalone strategies (step-by-step, verify, narrow) do not. This is a design choice, not a confound: follow-up strategies are explicitly designed to react to the model’s first answer. We report which strategies are standalone vs. follow-up in all results.

Abstention metrics. We report two metrics throughout: (1) *Accuracy*: fraction of all questions answered correctly (strategy applied, no abstention). (2) *Success rate*: fraction of questions where the model either answers correctly or correctly abstains (abstains on a question it would have answered wrongly). Success rate is always higher than accuracy because correct abstention counts as success. We report both metrics explicitly so readers can evaluate the abstention trade-off: higher success rate comes at the cost of leaving some questions unanswered. Incorrect abstention (abstaining on a question the model would have answered correctly) is implicitly penalised — it reduces the success rate.

Calibration is not held-out by default. All results in this paper are reported on held-out questions (calibration questions excluded from evaluation). We discovered during a pre-mortem analysis of our own paper that including calibration questions inflated lifts by 0–6 pp depending on the cell. The held-out split is essential for honest reporting.

Additional limitations: - **Logprobs required:** Not all APIs expose logprobs (notably, some commercial endpoints disable them). - **Calibration cost:** 50–200 questions × 8 strategies = 400–1600 API calls per new cell. - **Extra inference cost:** 1–4 additional calls per question at deployment time. - **Short responses:** CR is less informative for very short responses (1–2 tokens in MC), though it still works empirically. - **Model drift:** Together.ai updated the Cogito checkpoint between our calibration and deployment runs, changing vanilla accuracy by roughly 16–19 pp. We mitigate this in our reported results by measuring lift as paired within-question rescue (improvement against the model’s own vanilla answer in the same session), which is drift-robust. But this mitigation is partial, and the limitation is more serious than a single caveat sentence conveys. Three consequences are worth stating plainly. (i) *Cross-session reproducibility*: absolute accuracy numbers on cloud-served checkpoints are session-specific and will not reproduce exactly; only the paired within-question comparisons are stable. (ii) *Calibration transfer*: a calibration computed before a checkpoint update may no longer be optimal afterwards — strategy rankings and thresholds may need re-running. (iii) *Survivorship risk*: when a checkpoint update is suspected mid-experiment, paired rescue measured only on the post-update vanilla-wrong subset can be biased, because the post-update model fails a different set of questions than the pre-update model. We therefore recommend that calibration and deployment occur in the same session against a checkpoint whose identity is logged, and we report platform and approximate date for every cell. This is a property of commercial cloud-inference infrastructure that all LLM benchmarking must contend with; it is not specific to our method, but it does bound the reproducibility of any result reported against a cloud endpoint. - **GPT-oss-20B:** The smallest cell (n=148 held-out) does not reach statistical significance. We report it for completeness but do not include it in headline claims.

5.8 Optional: iterative adaptation as reflexive matching

This section is an optional theoretical aside. It interprets the empirical result of §5.1 — that follow-up strategies applied to already-answered questions outperform standalone strategies —

through the companion framework. None of the results in §4 or the recommendations in §5.1–§5.5 depend on it; a reader uninterested in the theory can skip to §6.

One empirical observation in §5.1 invites a theoretical reading: follow-up strategies (pre-mortem, re-examine) outperform standalone strategies when applied to already-answered questions. Under the companion framework (Lund 2026b §9.5), this is expected. There, the race architecture operates reflexively — what counts as “the task” the model is computing is itself an output of the model’s ongoing parse over the prompt, so there is no fixed situation whose optimal strategy could be read off in one shot. Iterative re-matching, in which each round’s intervention modifies what subsequent rounds condition on, is the framework-predicted shape of an effective pipeline. The empirical finding is consistent with this reading, and the clinical analogue is developed in a companion paper (Paper 8 §2.3). We present this as interpretation, not as evidence: the iterative-adaptive design is justified empirically by §4–§5.1 regardless of whether the reflexive-matching account is accepted.

6. Conclusion

We have shown that large language models’ logprob distributions contain a free, architecture-independent signal — competing routes — that serves three distinct functions at inference time: (1) calibrating which correction strategies help a given model, (2) identifying questions where the model should abstain rather than commit, and (3) detecting commitment gaps that can be recovered through re-prompting. On held-out data, a calibrated pipeline improves accuracy by +7.7 to +20.8 pp across four statistically significant cells spanning four architectures and four benchmarks (mean +11.8 pp). When abstention is permitted, CR-guided uncertainty thresholds improve success rates further at zero additional inference cost.

The method is practical: it requires no model training and no external verifier models. It does require a labelled calibration set of 50–200 questions per cell — that set, not the signal, is the real deployment cost. The signal itself is free to extract, and the pipeline adds 1–4 inference calls per question. The audit harness used to produce these results — automated pre-flight checks, oracle-plateau verification, oracle-cheat detection — is released with the code so the pipeline can be re-run with the same safeguards.

The companion paper (Lund 2026b) offers a theoretical account of why a logprob-derived signal should behave this way; readers interested in that account will find it there. The contribution of the present paper is narrower and self-contained: a calibrated inference-time pipeline, validated on five model–benchmark cells, that delivers measurable and statistically significant accuracy gains on the models we evaluated, using a signal that is free to extract and a calibration procedure that costs roughly a dollar and two hours per cell.

Acknowledgments

Claude (Anthropic, 2025–2026) acknowledged for research assistance. The theoretical claims, interpretations, and predictions are those of the author.

References

- Chen, L., Zaharia, M., Zou, J. (2023). How Is ChatGPT’s Behavior Changing over Time? *arXiv:2307.09009*.
- Cobbe, K. et al. (2021). Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*.
- Feng, S. et al. (2025). Know Your Limits: A Survey of Abstention in Large Language Models. *TACL*.
- Geng, J. et al. (2024). A Survey of Confidence Estimation and Calibration in Large Language Models. *NAACL 2024*.
- Grosse, R. et al. (2023). Studying Large Language Model Generalization with Influence Functions. *arXiv:2308.03296*.
- Huang, J. et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. *ICLR 2024*.
- Huang, L. et al. (2026). I-CALM: Incentivizing Confidence-Aware Abstention for LLM Hallucination Mitigation. *arXiv:2604.03904*.
- Kadavath, S. et al. (2022). Language Models (Mostly) Know What They Know. *arXiv:2207.05221*.
- Kamoi, R. et al. (2024). When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs. *TACL*.
- Klein, G. (1998). *Sources of Power: How People Make Decisions*. MIT Press.
- Kojima, T. et al. (2022). Large Language Models are Zero-Shot Reasoners. *NeurIPS 2022*.
- Li, Z. et al. (2025). Entropy-Guided Loop: Achieving Reasoning through Uncertainty-Aware Generation. *arXiv:2509.00079*.
- Liang, P. et al. (2022). Holistic Evaluation of Language Models. *arXiv:2211.09110* (HELM).
- Liu, A., Wu, Z., Michael, J., Suhr, A., West, P., Koller, A., Swayamdipta, S., Smith, N. A., & Choi, Y. (2023). We’re Afraid Language Models Aren’t Modeling Ambiguity. *EMNLP 2023*.
- Lightman, H. et al. (2023). Let’s Verify Step by Step. *arXiv:2305.20050*.
- Lu, Y. et al. (2022). Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. *ACL 2022*.
- Lund, T. (2026a). Behavioural Friction Theory: Toward a Common Currency for Behavioural Science. Zenodo. DOI: 10.5281/zenodo.19462500. [Paper 0, foundational]
- Lund, T. (2026b). Friction as the Cost of Probabilistic Computation: A Generalised Substrate Theory. Zenodo. DOI: 10.5281/zenodo.20012655. [Paper 1, companion theoretical paper]
- Lund, T. (2026c). Capacity Scaling of Encoding-Through-Loading: Application vs. Cloze Asymmetry Across Three Orders of Magnitude. Zenodo. DOI: 10.5281/zenodo.20013491. [Paper 2, companion empirical paper]
- Lund, T. (2026e). Cross-substrate replication of classical learning phenomena on LLM substrate. [Paper 4, in preparation]

- Lund, T. (2026, in preparation). A Recursive Strategy-Choice Race Account of Expertise Reversal. [Paper 4B; companion to Paper 4 with 7-experiment series on Qwen2-1.5B / Qwen2.5-7B / Llama-3.3-70B; cited in §5.4 for cross-substrate U-curve, strategy-commitment, and format-violation reactance findings.]
- Lyu, Q. et al. (2024). Learning to Route LLMs with Confidence Tokens. *arXiv:2410.13284*.
- Madaan, A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS 2023*.
- Mizrahi, M. et al. (2024). State of What Art? A Call for Multi-Prompt LLM Evaluation. *TACL*.
- Rein, D. et al. (2024). GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv:2311.12022*.
- Sclar, M. et al. (2024). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design, or: How I Learned to Start Worrying About Prompt Formatting. *ICLR 2024*.
- Snell, C. et al. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. *arXiv:2408.03314*.
- Straitouri, E. et al. (2025). TECP: Token-Entropy Conformal Prediction for LLMs. *Mathematics 13(20)*.
- Wang, X. et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023*.
- Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*.